

Técnicas Avanzadas de Minería de Datos

Métodos de Ensamble: La Sabiduría de las Multitudes (AdaBoost y XGBoost)

Ingeniería en Inteligencia Artificial

Instituto Politécnico Nacional

Daniel Sánchez Ruiz

Agenda de la Sesión

1. Introducción a los Métodos de Ensamble
2. La Filosofía del Boosting
3. AdaBoost: Adaptive Boosting
4. Desarrollo Paso a Paso: Calculando AdaBoost
5. XGBoost (Extreme Gradient Boosting)
6. Ejercicio Práctico y Solución
7. Implementación Computacional (Python)
8. Conclusiones y Referencias

¿Qué son los Métodos de Ensamble?

Ya conocemos métodos individuales como los Árboles de Decisión, es momento de avanzar. ¿Qué pasaría si en lugar de usar un solo modelo, usáramos cientos de ellos combinados?.

La Analogía Médica y "La Sabiduría de las Multitudes"

Imagina que tienes una enfermedad rara. Puedes ir con un solo médico (un modelo) y confiar en su diagnóstico, el cual podría estar sesgado. O puedes reunir un comité de 100 médicos con diferentes especialidades. Si promedias sus diagnósticos o los sometes a votación, el resultado será abrumadoramente más certero. Esto se conoce en la ciencia de datos como la "**Sabiduría de la Multitud**" (Wisdom of the Crowd).

Un ensemble combina múltiples **clasificadores base** (a menudo "débiles") para formar un modelo predictivo robusto y altamente preciso.

¿Por qué usar Ensamblados? (Sesgo vs Varianza)

Los beneficios matemáticos de usar un ensamble de modelos de clasificación en lugar de uno solo son principalmente dos:

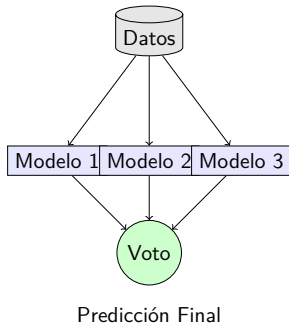
- 1 **Reducción de la Varianza:** Un modelo inestable (como un árbol muy profundo) cambia radicalmente si la muestra de datos cambia un poco. Al promediar muchos árboles (como en *Bagging*), la varianza se reduce drásticamente.
- 2 **Reducción del Sesgo (Error):** Algunos modelos son demasiado simples (como un tocón de decisión o árbol de profundidad 1) y tienen un alto margen de error. Al obligar a una secuencia de modelos a enfocarse en los errores pasados (como en *Boosting*), el ensamble general reduce su tasa de error.

Las dos grandes familias: Bagging vs Boosting

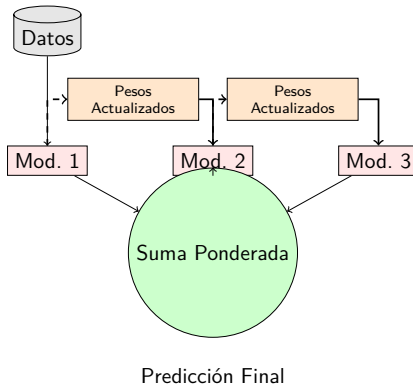
Característica	Bagging (ej. Random Forest)	Boosting (ej. AdaBoost, XGBoost)
Arquitectura	Paralela: Los modelos se entrenan simultáneamente e independientemente.	Secuencial: Cada modelo se entrena en un paso, aprendiendo del anterior.
Muestreo	Muestreo aleatorio con reemplazo uniforme (Bootstrap).	Los casos mal clasificados reciben mayor peso en la siguiente iteración.
Meta Principal	Reducir la Varianza (Evitar Overfitting).	Reducir el Sesgo (Mejorar precisión en modelos débiles).

Bagging vs Boosting

BAGGING (Paralelo)



BOOSTING (Secuencial)



El paradigma del Aprendiz Débil (Weak Learner)

Históricamente, Kearns y Valiant propusieron los conceptos de algoritmos 'fuertemente aprendibles' y 'débilmente aprendibles' en la teoría PAC (Probably Approximately Correct).

- **Aprendiz Fuerte:** Un modelo complejo (ej. Red Neuronal Profunda) que puede clasificar casi perfectamente la data por sí solo, pero requiere inmenso poder de cómputo.
- **Aprendiz Débil:** Un modelo extremadamente simple y rápido, cuya precisión es **solo ligeramente superior a adivinar al azar** (ej. $> 50\%$ en clasificación binaria).

El Teorema de Schapire: Demostró matemáticamente que un aprendizaje débil y uno fuerte son equivalentes. Un algoritmo débil **puede ser impulsado (boosted)** para convertirse en un algoritmo fuerte.

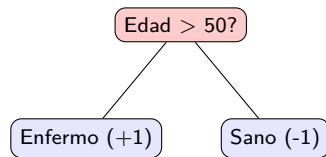
El Caballo de Batalla del Boosting: El Tocón de Decisión

El aprendiz débil más utilizado en algoritmos como AdaBoost es el **Tocón de Decisión (Decision Stump)**.

¿Qué es un tocón? Es un Árbol de Decisión con **una sola división** (profundidad = 1) y dos nodos hoja.

Evalúa solo **una variable** a la vez. Por ejemplo: "*Si $Edad > 50$, entonces Enfermo, sino Sano*".

Obviamente, cometerá muchos errores. Pero es baratísimo de calcular computacionalmente.



AdaBoost: Impulso Adaptativo

Propuesto por Freund y Schapire en 1995, **AdaBoost** resolvió el dilema de cómo combinar estos modelos débiles.

Responde a dos preguntas fundamentales:

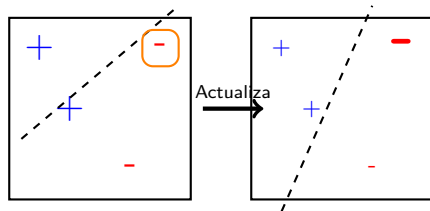
- 1 **¿Cómo cambiar los datos en cada ronda?** AdaBoost *aumenta el peso* (la importancia) de las muestras que el clasificador anterior clasificó mal, y *disminuye el peso* de las que clasificó bien.
- 2 **¿Cómo combinar los clasificadores finales?** Mediante una "Votación Mayoritaria Ponderada". A los clasificadores débiles que tuvieron menos errores se les otorga un mayor poder de voto (α).

Evolución Visual de los Pesos en AdaBoost

Observa cómo el tamaño (peso) de los signos mal clasificados (encerrados en círculos) crece en la siguiente iteración para forzar al modelo a prestarles atención.

Ronda 1 (G_1)

Ronda 2 (G_2)



En la Ronda 2, el clasificador crea una frontera radicalmente distinta solo para intentar no fallar con el signo menos "gigante".

Formulación Matemática de AdaBoost (1/2)

Dado un set de datos $T = \{(x_1, y_1), \dots, (x_N, y_N)\}$ con $y_i \in \{-1, +1\}$.

Paso 1: Inicialización de pesos

En la ronda inicial $m = 1$, todos los N pacientes valen lo mismo:

$$w_{1,i} = \frac{1}{N}, \quad i = 1, 2, \dots, N$$

Paso 2a: Entrenamiento y Error

Para la iteración m , se entrena el modelo $G_m(x)$ usando los pesos $w_{m,i}$. Se calcula el error ponderado ϵ_m :

$$\epsilon_m = \sum_{i=1}^N w_{m,i} \quad (\text{solo sumando donde } G_m(x_i) \neq y_i)$$

Formulación Matemática de AdaBoost (2/2)

Paso 2b: Importancia del modelo (α_m)

Calculamos el peso o "poder de voto" de este modelo. Mientras menor sea el error ϵ_m , mayor será su α_m :

$$\alpha_m = \frac{1}{2} \ln \left(\frac{1 - \epsilon_m}{\epsilon_m} \right)$$

Paso 2c: Actualización de pesos para los pacientes

Aumentamos el peso a los mal clasificados y lo disminuimos a los correctos:

$$w_{m+1,i} = \frac{w_{m,i}}{Z_m} \exp \left(-\alpha_m y_i G_m(x_i) \right)$$

(Donde Z_m es simplemente la suma de todos los nuevos pesos para normalizarlos y que sumen 1).

El Modelo Final: Un Ensamble Aditivo

Una vez que repetimos el proceso M veces (generando M tocones de decisión), la etapa de predicción para un nuevo paciente se vuelve sumamente simple.

Construimos una combinación lineal (Weighted Majority Vote) de todos los modelos básicos [17]:

$$f(x) = \sum_{m=1}^M \alpha_m G_m(x)$$

El clasificador fuerte final $G(x)$ simplemente evalúa el signo de esa suma:

Ecuación del Clasificador Fuerte Final

$$G(x) = \text{sign}(f(x)) = \text{sign} \left(\sum_{m=1}^M \alpha_m G_m(x) \right)$$

*Esta estructura es conocida teóricamente como el **Algoritmo Aditivo Escalonado Hacia Adelante** (Forward Stepwise Additive Model) minimizando la función de pérdida exponencial.*

Desarrollo Paso a Paso: Planteamiento del Problema (1/5)

Para comprender la mecánica subyacente, calculemos a mano la iteración 1 de AdaBoost para un set de datos médico de 5 pacientes con clases $Y \in \{+1, -1\}$.

Paciente (i)	Datos	Clase Real (y_i)
1	...	+1 (Sano)
2	...	+1 (Sano)
3	...	-1 (Enfermo)
4	...	-1 (Enfermo)
5	...	+1 (Sano)

Inicialización ($m = 1$): Dado que $N = 5$, asignamos un peso idéntico a todos:

$$w_1 = [0,2, 0,2, 0,2, 0,2, 0,2]$$

Desarrollo Paso a Paso: Cálculo del Error (2/5)

1. Entrenar Clasificador Base $G_1(x)$: Supongamos que entrenamos un tocón de decisión $G_1(x)$ y nos arroja las siguientes predicciones:

$$G_1(X_{pred}) = [+1, +1, +1, -1, +1]$$

Notamos que el **Paciente 3** fue clasificado incorrectamente (Se predijo $+1$, pero en realidad era $y_3 = -1$). Los otros 4 fueron correctos.

2. Calcular Error Ponderado (ϵ_1): Sumamos los pesos $w_{1,i}$ de los casos incorrectos. En este caso, solo falló el Paciente 3, cuyo peso actual es 0,2.

$$\epsilon_1 = 0,2$$

Desarrollo Paso a Paso: Importancia del Modelo (3/5)

3. Calcular la Importancia del Modelo (α_1): Usamos el error calculado ($\epsilon_1 = 0,2$) para determinar cuánto "poder de voto" tendrá este árbol en la ecuación final.

$$\alpha_1 = \frac{1}{2} \ln \left(\frac{1 - \epsilon_1}{\epsilon_1} \right)$$

$$\alpha_1 = \frac{1}{2} \ln \left(\frac{1 - 0,2}{0,2} \right)$$

$$\alpha_1 = 0,5 \ln \left(\frac{0,8}{0,2} \right)$$

$$\alpha_1 = 0,5 \ln(4) \approx 0,5(1,386) \approx \mathbf{0,693}$$

Un α positivo y alto indica que este modelo fue mejor que el azar y tendrá buen peso en la votación.

Desarrollo Paso a Paso: Actualización de Pesos (4/5)

4. Modificar los pesos para la Ronda 2: Formula: $w_{new} = w_{old} \times \exp(-\alpha_1 y_i G_1(x_i))$

A) Para los 4 pacientes CORRECTOS: $y_i = G_1(x_i)$, por lo que $y_i G_1(x_i) = 1$.

$$w_{new} = 0,2 \times \exp(-0,693 \times 1)$$

$$w_{new} = 0,2 \times \exp(-0,693)$$

$$w_{new} \approx 0,2 \times 0,5 = \mathbf{0,1}$$

El peso de los pacientes fáciles bajó a la mitad.

B) Para el Paciente 3 INCORRECTO: $y_3 \neq G_1(x_3)$, por lo que $(-1)(+1) = -1$.

$$w_{new} = 0,2 \times \exp(-0,693 \times -1) = 0,2 \times \exp(0,693)$$

$$w_{new} \approx 0,2 \times 2,0 = \mathbf{0,4}$$

El peso del paciente difícil se duplicó para que el siguiente árbol (G_2) se concentre en él.

Desarrollo Paso a Paso: Normalización (5/5)

5. Factor de Normalización (Z_1): Nuestros pesos crudos son: $[0,1, 0,1, 0,4, 0,1, 0,1]$. Para que sigan siendo una distribución de probabilidad válida, deben sumar 1 [17].

$$Z_1 = \sum w_{new} = 0,1 + 0,1 + 0,4 + 0,1 + 0,1 = \mathbf{0,8}$$

Dividimos cada peso entre Z_1 :

- Correctos: $w_2 = 0,1/0,8 = \mathbf{0,125}$
- Incorrecto: $w_2 = 0,4/0,8 = \mathbf{0,500}$

Nuevos pesos para la Ronda 2: $w_2 = [0,125, 0,125, \mathbf{0,500}, 0,125, 0,125]$.

El algoritmo ahora construirá a $G_2(x)$, el cual estará obsesionado con no equivocarse con el paciente 3, ya que representa el 50 % de todo el peso del dataset.

Limitaciones de AdaBoost

A pesar de su elegancia matemática, AdaBoost tiene defectos críticos en entornos de *Big Data* corporativos modernos:

La vulnerabilidad al ruido (Overfitting)

Si nuestro dataset tiene anomalías (outliers) o datos mal etiquetados por error humano, AdaBoost les asignará pesos inmensos, persiguiendo imposibles y **sobreajustando** (overfitting) el modelo fuertemente a ese ruido.

Además, AdaBoost no fue optimizado para sistemas distribuidos y se ejecuta lentamente en conjuntos de datos de millones de registros.

La evolución natural de esta familia llegó en 2014 con **XGBoost**.

XGBoost: Extreme Gradient Boosting

XGBoost fue introducido por Chen y Guestrin en 2014 y rápidamente se convirtió en el Estado del Arte en Machine Learning para datos tabulares.

¿Qué lo hace tan superior?

- ➊ **Regularización Matemática L1/L2:** A diferencia de AdaBoost, la función objetivo de XGBoost incluye términos de penalización (shrinkage/elastic net) que restringen la complejidad de los árboles, lo que virtualmente erradica el overfitting en datos ruidosos.
- ➋ **Manejo nativo de valores faltantes (Sparsity):** Aprende la dirección por defecto si falta un dato.
- ➌ **Exact Greedy Algorithm:** Su algoritmo puede enumerar en milisegundos todas las posibles divisiones de las variables en los nodos.

Dominio en el Mundo Real: Competencias y Escalabilidad

El rey de Kaggle Antes de la explosión del Deep Learning en 2016, **casi todas** las competencias en la plataforma Kaggle eran ganadas utilizando XGBoost.

Escalabilidad Extrema (Cache-aware) A diferencia del software antiguo, XGBoost fue construido para explotar paralelismo de CPU multinúcleo y optimización de memoria caché (*cache-aware*), escalando eficientemente para procesar billones de registros en clústeres.

Ejemplos Reales

- Premio *Netflix* para sistemas de recomendación.
- Detección en tiempo real de videos manipulados (*DeepFakes*) que amenazan la privacidad.

Ejercicio Práctico para los Alumnos (AdaBoost Iteración 2)

Contexto: Retomemos el problema del paso a paso. Hemos finalizado la Ronda 1 de AdaBoost y entramos a la Ronda 2.

Nuestra nueva distribución de pesos normalizada (w_2) para los 5 pacientes es:

$$w_2 = [0,125, 0,125, \mathbf{0,500}, 0,125, 0,125]$$

Entrenamos un nuevo Tocón de Decisión $G_2(x)$, el cual, enfocado en corregir al Paciente 3 (que vale el 50 % del peso), clasifica perfectamente a los pacientes 3, 4 y 5, pero se equivoca en los pacientes 1 y 2.

Instrucciones:

- 1 Calcula el Error Ponderado ϵ_2 del modelo G_2 .
- 2 Calcula el Poder de Voto α_2 de este segundo modelo. (Usa $\ln(3) \approx 1,098$).
- 3 Sabiendo que el Paciente 3 fue clasificado **correctamente** esta vez, calcula su peso no-normalizado (w_{new}) de cara a la Ronda 3.

Solución al Ejercicio (Paso 1 y 2)

1. Error Ponderado (ϵ_2): Los únicos pacientes mal clasificados por G_2 fueron el paciente 1 y el paciente 2. Sus pesos en esta ronda (w_2) son 0,125 cada uno.

$$\epsilon_2 = 0,125 + 0,125 = \mathbf{0,25}$$

2. Poder de Voto del Modelo (α_2): Utilizamos la fórmula del logaritmo:

$$\begin{aligned}\alpha_2 &= \frac{1}{2} \ln \left(\frac{1 - \epsilon_2}{\epsilon_2} \right) \\ \alpha_2 &= 0,5 \ln \left(\frac{1 - 0,25}{0,25} \right) = 0,5 \ln \left(\frac{0,75}{0,25} \right) \\ \alpha_2 &= 0,5 \ln(3) \approx 0,5(1,098) \approx \mathbf{0,549}\end{aligned}$$

Solución al Ejercicio (Paso 3)

3. Nuevo Peso del Paciente 3: El Paciente 3 **fue clasificado correctamente** en esta ronda. Por lo tanto $y_3 G_2(x_3) = 1$. Su peso actual al arrancar la ronda era $w_{2,3} = 0,500$.

Aplicamos la ecuación de actualización (disminución):

$$w_{new} = w_{2,3} \times \exp(-\alpha_2 \times 1)$$

$$w_{new} = 0,500 \times \exp(-0,549)$$

Sabemos que $\exp(-0,549) \approx 0,577$.

$$w_{new} \approx 0,500 \times 0,577 \approx \mathbf{0,288}$$

Conclusión: Dado que el modelo logró resolver el caso difícil del Paciente 3, su influencia vuelve a bajar para permitir que el algoritmo se concentre en los otros pacientes.

Implementación en Python

Ejemplo mínimo de entrenamiento con XGBoost en Python.

```
import pandas as pd
import xgboost as xgb
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Cargar datos tabulares
df = pd.read_csv("datos_medicos.csv")
X = df.drop('Enfermedad', axis=1)
y = df['Enfermedad']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Inicializar modelo XGBoost (Gradient Boosting Extremo)
# n_estimators = cantidad de iteraciones secuenciales M
modelo_xgb = xgb.XGBClassifier(n_estimators=100, learning_rate=0.1, max_depth=3)

# Entrenar modelo
modelo_xgb.fit(X_train, y_train)

# Predecir
predicciones = modelo_xgb.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, predicciones)}")
```

Debate Final: ¿Cuál es mejor en la vida real?

La decisión entre usar Bagging (ej. Random Forest) o Boosting (ej. XGBoost) ha sido el núcleo de intensos debates [8].

- En datasets con **muchísimo ruido**, outliers o errores en las etiquetas, Random Forest suele ser más seguro, ya que XGBoost/AdaBoost tenderá a perseguir el ruido y sobreajustarse.
- En datasets **relativamente limpios**, el Boosting (particularmente XGBoost) dominará abrumadoramente en términos de precisión predictiva.
- **Trade-off:** Bagging es excelente minimizando la varianza, Boosting minimizando el sesgo. Idealmente, un científico de datos competitivo entrenará ambos y comparará empíricamente.

Resumen de la Sesión

- Reemplazamos la idea de crear un único modelo hiper-complejo (como una red neuronal) por la brillante noción de **Ensamblar** múltiples modelos ultra-simples para formar "La Sabiduría de la Multitud".
- La técnica **Bagging** trabaja en paralelo, basándose en la variabilidad del muestreo (Bootstrap).
- La técnica **Boosting** trabaja en secuencia. Algoritmos como **AdaBoost** transforman "aprendices débiles" penalizando iterativamente a las tuplas mal clasificadas mediante pesos matemáticos y exponenciales.
- **XGBoost** representa el estado del arte de esta familia de métodos, agregando paralelismo eficiente, manejo de datos ausentes y regularizaciones L1/L2 para evitar el sobreajuste.

Referencias Bibliográficas I

- Li, H. (2024). *Machine Learning Methods*. Tsinghua University Press & Springer. (Secciones de Boosting, The Basic Idea of Boosting, AdaBoost Algorithm, pp. 179-182) [3, 11, 17, 24].
- Yu, C.H. (2022). *Data Mining and Exploration: From Traditional Statistics to Modern Data Science*. CRC Press. (Capítulo 7: Ensemble Methods: The Wisdom of the Crowd, XGBoost vs AdaBoost, pp. 159-160) [2, 6, 10, 20].
- Han, J., Pei, J., & Tong, H. (2022). *Data Mining: Concepts and Techniques* (4th ed.). Morgan Kaufmann. (Capítulo 6: Gradient Tree Boosting para Regresión).

Referencias Bibliográficas II

- Larose, D. T., & Larose, C. D. (2015). *Data Mining and Predictive Analytics* (2nd ed.). Wiley. (Capítulo 25: Ensemble Methods: Bagging and Boosting) [1, 4, 5, 25].
- Freund Y., & Schapire R.E. (1995). A decision-theoretic generalization of on-line learning and an application to boosting. *Computational learning theory. EuroCOLT 1995*. Springer.
- Chen T., & Guestrin C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.

¿Preguntas o Comentarios?

Fin de la presentación